10주차_6장

6.0 텍스트 벡터화 개요

컴퓨터는 텍스트를 직접 이해하지 못함 → 텍스트 벡터화(text vectorization), 텍스트를 숫자로 변환하는 과정 필요

기초적인 벡터화

- 원-핫 인코딩(One-hot Encoding): 각 단어에 고유 인덱스 매핑, 해당 위치만 1, 나머지 0
- 빈도 벡터화(Count-Vectorization): 단어 등장 횟수를 값으로 사용

문제점

1. 벡터가 희소하고 차원 크다 → 컴퓨팅 비용 증가, 차원의 저주
2. 의미를 담지 못함

해결책

- 워드 임베딩(Word Embedding): 단어를 고정 길이 실수 벡터로 표현. 벡터 공간에서의 상대 위치로 단어 간 관계 추론(Word2Vec, fastText)
- 워드 임베딩은 단어마다 임베딩 고정 → 다의어, 문맥을 다루기 어려움 ⇒ 신경망을 활용한 동적 임베딩(Dynamic Embedding)

6.1 언어 모델

언어 모델: 입력된 문장으로 다음 문장, 단어를 생성할 수 있도록 단어 시권스에 확률을 할당하는 모델. 번역, 음성 인식, 요약에 쓰임

완성된 문장 단위로 확률 계산하는 것은 사실상 불가능 → 토큰 하나씩 예측하는 자기회귀 방식 고안

자기회귀 언어 모델(Autoregressive LM)

이전 단어들이 주어졌을 때 다음 단어의 조건부 확률 계산

$$P(w_t|w_1, w_2, ..., w_{t-1})$$

조건부 확률의 연쇄 법칙(chain rule) 적용하면 문장 전체의 확률 구할 수 있음

$$P(w_1, ..., w_t) = P(w_1)P(w_2|w_1)...P(w_t|w1, ...w_{t-1}) = \prod_{i=1}^{t} P(w_i \mid w_{<i})$$

모델의 출력이 다시 입력으로 들어가는 구조 → "자기 회귀"

각 시점에서 다음 토큰 고르는 것 - 토큰 분류 문제

통계적 언어 모델(Statistical LM)

언어의 통계적 구조로 시퀀스 확률 추정. 마르코프 체인(Markov Chain)으로 구현

e.g.) 안녕하세요 1000번 중 안녕하세요 만나서 700번, 안녕하세요 반갑습니다 100번인 경우

$$P(\text{만나서}|\text{안녕하세요}) = \frac{700}{1000}$$

$$P(\text{반갑습니다}|\text{안녕하세요}) = \frac{100}{1000}$$

- 장점: 대규모 데이터 효과적으로 처리
- 단점: 한 번도 등장하지 않은 단어, 문장은 확률 0 ⇒ 데이터 희소성(data sparsity) 문제

마르코프 가정과 N-gram의 연결

연쇄법칙 마지막 항

$$P(w_t|w_1, ..., w_{t-1})$$

바로 앞 N-1개의 단어만 본다

$$P(w_t \mid w_1, \ldots, w_{t-1}) \approx P(w_t \mid w_{t-N+1}, \ldots, w_{t-1})$$

$$\hat{P}(w_t \mid w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t)}{\text{count}(w_{t-1})}$$

사전 학습과 언어 모델

최근 모델은 언어 모델링으로 가중치 사전 학습함

- GPT: 인과적 언어 모델; "GPT는 OpenAI에서 개발한" → 다음 단어 '인과적' 예측 ⇒ 이어서 '언어 모델입니다' 예측
- BERT: 마스킹된 언어 모델: "BERT는 [MASK]에서 개발한 [MASK]언어 모델입니다." 'Google', '마스킹된' 예측

6.1 N-gram

연속된 N개 단어를 하나의 단위로 묶어 다음 단어 확률 추정

N	이름	"안녕하세요 만나서 진심으로 반가워요"
1	유니그램	안녕하세요 / 만나서 / 진심으로 / 반가워요
2	바이그램	안녕하세요 만나서 / 만나서 진심으로 / 진심으로 반가워요
3	트라이그램	안녕하세요 만나서 진심으로 / 만나서 진심으로 반가워요
≥4	N-gram	

$$P(w_t \mid w_{t-1}, w_{t-2}, \ldots, w_{t-N+1})$$

N값으로 모델 복잡도 조절, 파이썬 zip 또는 nltk.ngrams로 구현 가능

zip(*[words[i:] for i in range(n)]) 동작 원리

n=2이면 zip(words[0:], words[1:]) → (w0, w1), (w1, w2), ... 처럼 한 칸씩 밀린 리스트 나란히 묶음
zip은 가장 짧은 리스트에서 멈춤 → 끝부분 자동으로 처리 됨

N-gram의 딜레마

- N이 크면 문맥을 더 보지만 해당 N gram이 말뭉치에 거의 없음 → 희소성이 심해짐
- N이 작아지면 희소성은 줄지만 긴 문맥은 보지 못함

한계를 넘기 위해 신경망 언어 모델(RNN→ 트랜스포머) 등장

```
import nltk



def ngrams(sentence, n):
    words = sentence.split()
    ngrams = zip(*[words[i:] for i in range(n)])
    return list(ngrams)


sentence = "안녕하세요 만나서 진심으로 반가워요"

unigram = ngrams(sentence, 1)
bigram = ngrams(sentence, 2)
trigram = ngrams(sentence, 3)

print(unigram)
print(bigram)
print(trigram)
```

```
unigram = nltk.ngrams(sentence.split(), 1)
bigram = nltk.ngrams(sentence.split(), 2)
trigram = nltk.ngrams(sentence.split(), 3)

print(list(unigram))
print(list(bigram))
print(list(trigram))
```

6.3 TF-IDF

TF-IDF(Term Frequency-Inverse Document Frequency): 문서 안에서 특정 단어가 얼마나 중요한지 계산하는 통계적 가중치

BoW에 가중치 부여

BoW(Bag of Words)

문서를 단어의 집합(가방)으로 표현, 중복 허용해 빈도 기록

원핫은 등장 여부만, BoW는 빈도까지

	I	like	this	movie	don't	famous	is
This movie is famous movie	0	0	1	2	0	1	1
I like this movie	1	1	1	1	0	0	0
I don't like this movie	1	1	1	1	1	0	0

모든 단어 가중치 같음

긍정/부정 분류에서 결정적 단서 - don't인데 자주 나오지 않아 무시될 수 있음

수식

단어 빈도(TF): 문서 d에서 단어 t가 나온 횟수

$$TF(t,d) = count(t,d)$$

TF 높으면 문서에서 중요한 단어일수도 있지만, 원래 자주 쓰이는 단어일수도 있음. 문서 길수록 TF 커짐

문서 빈도(DF): 단어 t가 등장한 문서의 수

$$DF(t,D) = \text{count}(t \in d : d \in D)$$

DF가 높으면 흔한 단어 → 중요도 낮음, DF가 낮으면 특정 문맥에서만 쓰여 중요도 높을 수 있음

역문서 빈도(IDF): 전체 문서 수를 DF로 나누고 로그 취함

$$IDF(t,D) = \log\left(\frac{\text{count}(D)}{1 + DF(t,D)}\right)$$

- 분모 +1: 한 번도 등장하지 않은 단어에서 분모 0 되는 것을 방지
- log: 값이 폭주하지 않도록 로그로 완만하게 만듦

TF-IDF

$$TF\text{-}IDF(t,d,D) = TF(t,d) \times IDF(t,D)$$

문서 안에서는 자주, 전체에서는 드물게 나오는 단어의 값이 커짐
관사와 같이 어디에나 나오는 단어의 가중치는 낮아짐

사이킷런

```
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = [
    "That movie is famous movie",
    "I like that actor",
```

```
    "I don't like that actor"
]

tfidf_vectorizer = TfidfVectorizer()
tfidf_vectorizer.fit(corpus)
tfidf_matrix = tfidf_vectorizer.transform(corpus)

print(tfidf_matrix.toarray())
print(tfidf_vectorizer.vocabulary_)
```

```
# (추가) 사이킷런 공식으로 직접 계산해서 결과가 같은지 확인
import numpy as np
from collections import Counter

vocab_ = tfidf_vectorizer.vocabulary_
docs = [[w for w in tfidf_vectorizer.build_analyzer()(d)] for d in corpus]
n = len(docs)

df = Counter(w for d in docs for w in set(d))
idf = {w: np.log((1 + n) / (1 + df[w])) + 1 for w in vocab_}   # smooth_idf=True

manual = np.zeros((n, len(vocab_)))
for i, d in enumerate(docs):
    for w, c in Counter(d).items():
        manual[i, vocab_[w]] = c * idf[w]
manual = manual / np.linalg.norm(manual, axis=1, keepdims=True)  # L2 정규화

print(np.allclose(manual, tfidf_matrix.toarray()))
print("문서별 상위 3개 단어:", [[k for k, v in sorted(vocab_.items(), key=lambda x: x[1]) if v in ro
w]
                          for row in np.argsort(-manual, axis=1)[:, :3]])
```

- fit()으로 학습 → transform()으로 변환, toarray()로 넘파이 배열, vocabulary_로 단어 → 인덱스 사전 확인

참고